

DSA: 10 Python Solutions

Copy-Paste Ready Solutions

Q1. Product of Array Except Self

```
from typing import List

class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        answer = [1] * n

        prefix = 1
        for i in range(n):
            answer[i] = prefix
            prefix *= nums[i]

        suffix = 1
        for i in range(n - 1, -1, -1):
            answer[i] *= suffix
            suffix *= nums[i]

        return answer
```

Time: $O(n)$ | Extra Space: $O(1)$, excluding the output array

Q2. Longest Consecutive Sequence

```
from typing import List

class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        num_set = set(nums)
        longest = 0

        for num in num_set:
            if num - 1 not in num_set:
                current = num
                length = 1

                while current + 1 in num_set:
                    current += 1
                    length += 1

                longest = max(longest, length)

        return longest
```

Time: $O(n)$ average | Space: $O(n)$

Q3. 3Sum

```
from typing import List

class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        result = []

        for i in range(len(nums) - 2):
            if i > 0 and nums[i] == nums[i - 1]:
                continue

            left = i + 1
```

```

right = len(nums) - 1

while left < right:
    total = nums[i] + nums[left] + nums[right]

    if total == 0:
        result.append([nums[i], nums[left], nums[right]])
        left += 1
        right -= 1

        while left < right and nums[left] == nums[left - 1]:
            left += 1

        while left < right and nums[right] == nums[right + 1]:
            right -= 1

    elif total < 0:
        left += 1
    else:
        right -= 1

return result

```

Time: $O(n^2)$ | Extra Space: $O(1)$, excluding sorting and output

Q4. Daily Temperatures

```

from typing import List

class Solution:
    def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
        n = len(temperatures)
        answer = [0] * n
        stack = []

        for i in range(n):
            while stack and temperatures[i] > temperatures[stack[-1]]:
                prev = stack.pop()
                answer[prev] = i - prev

            stack.append(i)

        return answer

```

Time: $O(n)$ | Space: $O(n)$

Q5. Kth Largest Element in an Array

```

from typing import List

class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        nums.sort(reverse=True)
        return nums[k - 1]

```

Time: $O(n \log n)$ | Extra Space: $O(1)$

Q6. Merge K Sorted Linked Lists

```

from typing import List, Optional
import heapq

class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        heap = []

```

```

for i, node in enumerate(lists):
    if node:
        heapq.heappush(heap, (node.val, i, node))

dummy = ListNode(0)
current = dummy

while heap:
    value, i, node = heapq.heappop(heap)

    current.next = node
    current = current.next

    if node.next:
        heapq.heappush(heap, (node.next.val, i, node.next))

return dummy.next

```

Time: $O(N \log k)$ | Space: $O(k)$

Q7. Course Schedule II

```

from typing import List
from collections import deque

class Solution:
    def findOrder(self, numCourses: int,
                  prerequisites: List[List[int]]) -> List[int]:
        graph = [[] for _ in range(numCourses)]
        indegree = [0] * numCourses

        for a, b in prerequisites:
            graph[b].append(a)
            indegree[a] += 1

        queue = deque()

        for i in range(numCourses):
            if indegree[i] == 0:
                queue.append(i)

        order = []

        while queue:
            course = queue.popleft()
            order.append(course)

            for next_course in graph[course]:
                indegree[next_course] -= 1

                if indegree[next_course] == 0:
                    queue.append(next_course)

        if len(order) == numCourses:
            return order

        return []

```

Time: $O(V + E)$ | Space: $O(V + E)$

Q8. Lowest Common Ancestor of a Binary Tree

```

class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode',
                              p: 'TreeNode',
                              q: 'TreeNode') -> 'TreeNode':
        if root is None or root == p or root == q:
            return root

```



```
        right = mid

    if left == len(dp):
        dp.append(num)
    else:
        dp[left] = num

    return len(dp)
```

Time: $O(n \log n)$ | Space: $O(n)$